

Version 2.0

January 2026



OBJECT-ORIENTED PROGRAMMING

MODULE 2 - JAVA OOP (CLASS, METHOD, ATTRIBUTE)

Author:

Ir. Galih Wasis Wicaksono, S.Kom., M.Cs.

Yossua Agung Budianto

Muhammad Budi Kusuma

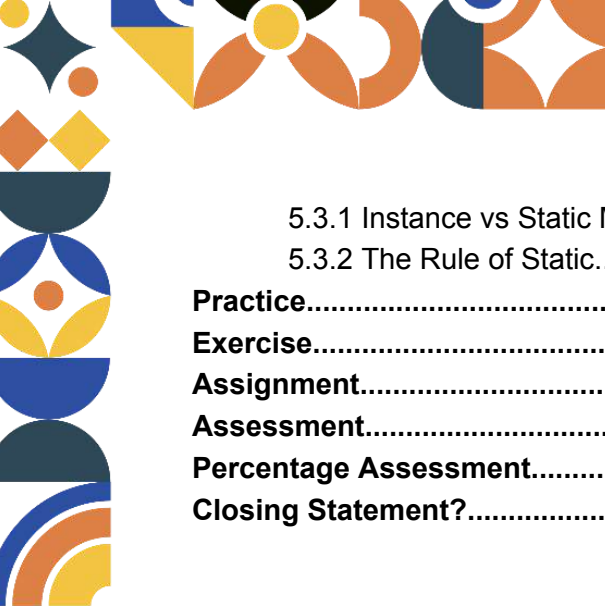
INFORMATICS LABORATORY

University of Muhammadiyah Malang

TABLE OF CONTENTS

TABLE OF CONTENTS.....	2
INTRODUCTION.....	4
OBJECTIVES.....	4
MODULE TARGETS.....	4
PREPARATION.....	4
KEYWORDS.....	4
1. OOP.....	5
1.1 Why Should I Understand This?.....	5
1.2 Formal Definition.....	5
2. Class.....	6
2.1 Why Should I Understand This?.....	6
2.2 Formal Definition.....	6
2.3 Concept Elaboration.....	7
2.3.1 Class as a Data Type.....	7
2.3.2 Class as a Blueprint.....	7
2.3.3 Class as a Logical Structure (Language Level).....	7
2.3.4 Array of Objects.....	8
2.3.5 Encapsulation.....	8
Public.....	9
Protected.....	9
Private.....	9
3. Attribute.....	10
3.1 Why Should I Understand This?.....	10
3.2 Formal Definition.....	10
3.3 Concept Elaboration.....	11
3.3.1 Anatomy of an Attribute (Declaration).....	11
3.3.2 Concept of “Default Value”.....	12
3.3.4 Analogy “Sticky Note on Cup of Coffee”.....	13
4. Method.....	13
4.1 Why Should I Understand This?.....	13
4.2 Formal Definition.....	14
4.3 Concept Elaboration.....	14
4.3.1 Anatomy of a Method.....	14
4.3.2 Method Overloading (Polymorphism Intro).....	15
4.3.3 Getter Setter Method.....	15
5. Static Method and Static Attribute.....	18
5.1 Why Should I Understand This?.....	18
5.2 Formal Definition.....	18
5.3 Concept Elaboration.....	18





5.3.1 Instance vs Static Memory.....	18
5.3.2 The Rule of Static.....	18
Practice.....	19
Exercise.....	23
Assignment.....	25
Assessment.....	26
Percentage Assessment.....	26
Closing Statement?.....	27



INTRODUCTION

OBJECTIVES

1. To enable students to understand the fundamental of OOP in Java
2. To provide students with a clear understanding about classes, methods, and attributes

MODULE TARGETS

1. Students are able to create classes, methods, and attributes
2. Students are able to design a program based on object-oriented principles.

PREPARATION

1. Muslim students may recite the following prayer before starting the lesson
رَضِيتُ بِاللّٰهِ رَبًّا وَبِالْإِسْلَامِ دِينًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُولًا. رَبِّ زِدْنِي عِلْمًا وَارْزُقْنِي فَهْمًا
2. Laptop or personal computer
3. IntelliJ IDE/Any Java IDE (Eclipse /VSCode)
4. "Full Poweeeeerrrrrrrr"

KEYWORDS

Class, Method, Static, Attribute, getter, setter

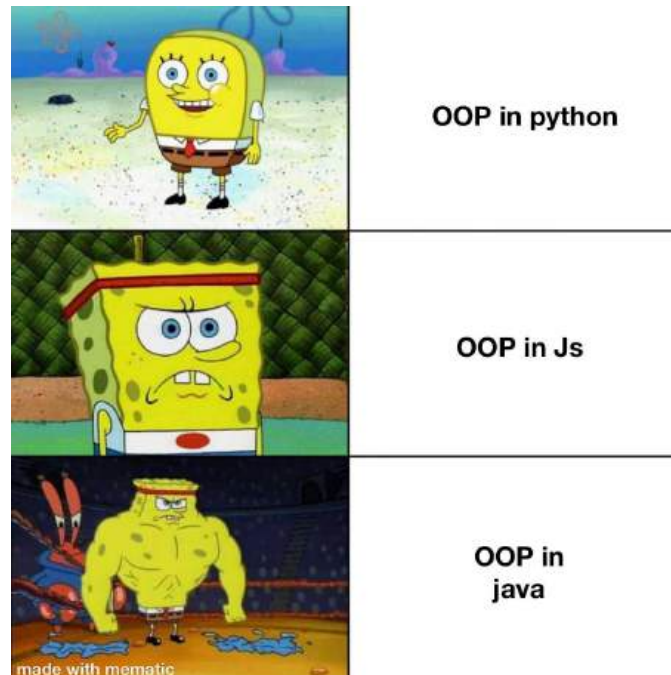


1. OOP

1.1 Why Should I Understand This?

Getting a handle on Object-Oriented Programming (OOP) isn't just about picking a style; it's a huge deal for building modern software that lasts and can grow. Its main power is how it totally reorganizes code into smaller, manageable chunks. This leads to big wins: your code is cleaner, way more organized, and much easier to read. By bundling data and actions into separate "objects," the codebase becomes simpler to grasp. Each object has a clear job, which seriously cuts down on the headache when you're debugging or adding new stuff. Plus, the core ideas give you a solid structure: Inheritance, Polymorphism, Encapsulation, and Abstraction. They let you break the app into a network of interacting pieces that act like real-world things, making the logic intuitive and easy to manage, especially when projects get huge.

1.2 Formal Definition



OOP, or Object-Oriented Programming (Pemrograman Berorientasi Objek) is a programming paradigm that focuses on objects. An object is an entity that combines data and functions. OOP helps make our programs more structured, readable, and easy to modify. Programs built with OOP are a combination of some small components, which makes it easier for programmers to maintain or expand the program. Objects that are related to each other are grouped together in a class, then these objects interact with each other to solve complex problems.



2. Class

2.1 Why Should I Understand This?



Imagine you're making a program for a movie theater. The program must contain information like the film name, the director's name, the ticket price, and so on. The program also needs to show the movie schedule and handle popcorn orders. You can't just write the program line by line like `String movie1 = "Jurassic Park"; int price1 = 20000;` and so on, it's inefficient, what if there are a hundred movies, or even a thousand? It would take a very long time and be very hard to read. That's why you need to learn about classes.

2.2 Formal Definition



A class is a fundamental concept of **OOP (Object-Oriented Programming/Pemrograman Berorientasi Objek)** that is used as a “blueprint” to create objects, making programs more structured and efficient. The concept of classes in programming languages is very broad. In Java, a class contains attributes (variables) and methods (functions), which we will discuss later. Attributes and methods define the specific characteristics and behaviors of the objects we create.

A class consists of **variables** called **attributes** or **properties**, and **functions** called **methods**. **A class** is different from a **function**, a class doesn't need parentheses () after its name. Instead, it goes directly to the class body, which starts with { and ends with }.

2.3 Concept Elaboration

What actually is a class in Java? A class is a new type definition that you create yourself, not just a template for objects, but a formal structure that the JVM recognizes. A class contains:

- Metadata (name, package, visibility)
- Internal Structure (attributes, methods, constructors, etc.)
- Hierarchy information (superclass, interfaces)
- Runtime information (annotations, modifiers)
- Compiled bytecode

For now let's focus on the basics of classes, we'll discuss the other aspects later.

2.3.1 Class as a Data Type

In java, a class is a data type that you define yourself. Built-in data types in Java include `int`, `char`, and `ArrayList`. But when you create a class, you're making your own custom data type.

2.3.2 Class as a Blueprint

As we discussed earlier, a class is a blueprint, and an object is the result created from that class. The class describes what the object will be like, not the object itself.

2.3.3 Class as a Logical Structure (Language Level)

In Java, each class has a unique name, modifiers, and a category. There are many categories of classes, such as: concrete class, abstract class, final class, sealed class, nested class, static nested class, inner class and so on. If you want to learn more about classes as logical structures, you can click [here](#).



2.3.4 Array of Objects

You can instantiate an object like this:

```
1 Student yossua = new Student();
```

You can also instantiate objects using an array. An array of objects is different from a regular array. Example:

```
1 Student[] kelasPBO = new Student[6];
```

We have declared an array for 6 Student objects, but the array elements are null, there are no actual Student objects in it yet.

2.3.5 Encapsulation

Encapsulation is a wrapper (pembungkus). Encapsulation in OOP means wrapping class methods or attributes to keep them within the class, so they can't be accessed by other classes. Encapsulation is implemented using access modifiers. There are 4 types of access modifiers: `private`, `public`, `protected` and default (no modifier). Access modifiers can be applied not just to attributes and methods, but also to interfaces, enums, and classes themselves.

The function of access modifiers in java is to limit scope from classes, constructors and other data members.

Modifier	Class	Package	Subclass	World
<code>public</code>	✓	✓	✓	✓
<code>protected</code>	✓	✓	✓	✗
<code>no modifier*</code>	✓	✓	✗	✗
<code>private</code>	✓	✗	✗	✗



Take a look at this code:

```

1 public class ModulePractice {
2     public static void main (String [] args){
3         Private String name;
4         public int age;
5         protected boolean gender; //male or female
6     }
7

```

The code marked above shows modifiers. Modifiers are used to control access to attributes, methods and classes.

Public

Gives access to any attribute or method so it can be accessed by anyone (any property or class, whether related or not). This means methods and attributes in a class can be accessed by everyone, both the class itself and other classes.

Protected

Gives access to the class itself and its subclasses (through inheritance). This means everything in the class can be accessed by the class itself and any class that extends it. Note that classes in the same package can also access protected members. Protected can only be used on attributes and methods, it cannot be used on classes, enums, or interfaces.

Private

Only gives access to the class itself. This means methods and attributes can only be accessed within the same class. This modifier cannot be used on classes, enums, or interfaces, only on attributes and methods.



3. Attribute

3.1 Why Should I Understand This?



Imagine you're handling the cashier system at a movie theater. When a customer orders a ticket for the movie "Dr.Stone : New World", a popcorn and a Coke, you can't just process it without storing information. The system must remember: Who's ordering? How many tickets are they ordering? What size Coke did they order? What type of popcorn did they and their girlfriend order? If we only use local variables (primitive variables), the data will be lost after the method finishes running. We need a storage place that stays with the object as long as the object exists in memory. That's the function of attributes. Without attributes, all objects would look the same and wouldn't have unique identification.

3.2 Formal Definition

Attribute or **properties** is an **identity data** or **information** from a class that we create, or what we know as **variables** in **C**. Here's what you need to know about **attributes**:

- An **attribute** is data or a property of a class.
- **Attributes** can be variables like **name**, **age**, **color**, and so on.
- **Attributes** define the state of an **object**.

Let's say we have a car with specifications like brand, year, speed, and color. Then we can write a class named Car like this:



```
public class Car {
    String merk;
    int year;
    String speed;
    String color;
}
```

3.3 Concept Elaboration

3.3.1 Anatomy of an Attribute (Declaration)

Declaration of an attribute is very crucial in Java, especially since Java is a case-sensitive programming language. The anatomy of an attribute is as follows:

```
[Access Modifier] [Data Type] [variable name] = [Initial value];
```

Let's discuss the details :

- Access Modifier: Controls access to a variable, method, or class. We have 4 types of access modifiers: public, protected, default (no modifier), and private.
- Data Type: The data type of the variable you want to use, like **String**, **int**, **char**, and so on.
- Variable Name: the name of the variable you need, like **name**, **age**, **price** and so on.
- Initial Value: The value you want to assign, like "**Yossua Agung**", "**Budi Kusuma**", **20**, **3.14** and so on.

Unlike local variables that live in the Stack (temporary), attributes live in the Heap (with its object). As long as the object isn't deleted, the data is preserved.



3.3.2 Concept of "Default Value"

There are some differences between attributes and local variables. Local variables must be initialized, if not, the compiler will throw an error. Example:

```

1 public class ModulePractice {
2     public static void main (String [] args){
3         String kelas;
4     }
5 }

```

As shown in the example above, if you try to use the variable without initializing it, you'll get a compile error. Here's the correct example:

```

1 public class ModulePractice {
2     public static void main (String [] args){
3         String kelas = "Kelasking";
4     }
5 }

```

Meanwhile, if an attribute isn't initialized, Java will assign a default value automatically. Here are the default values for attributes:

Data Type	Default Value
byte	0
short	0
int	0



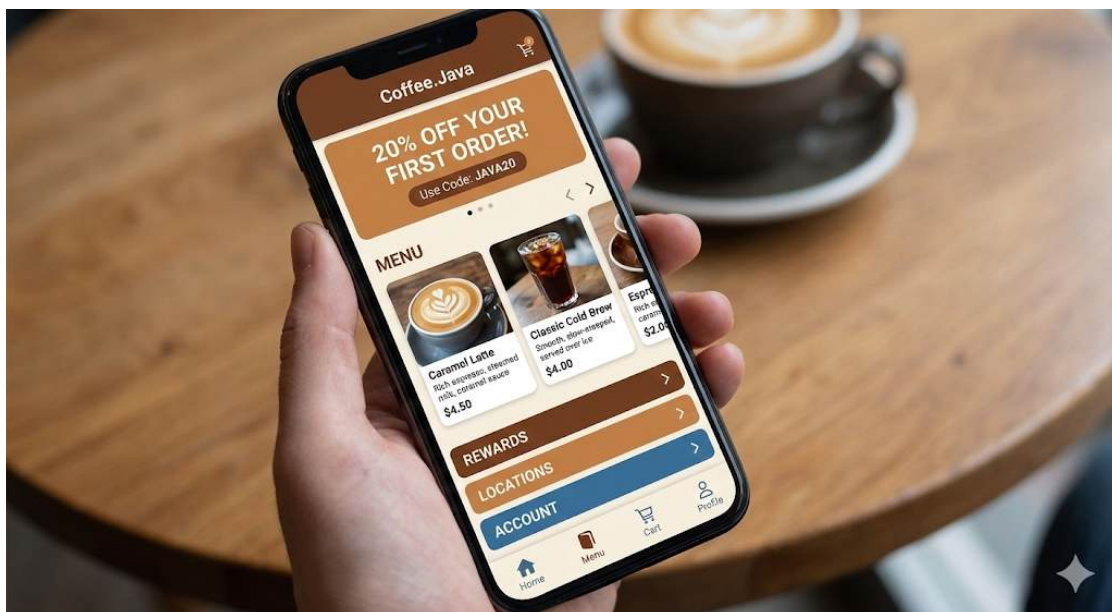
long	0L ▾
float	0.0f ▾
double	0.0d ▾
char	\u0000 (null character) ▾
boolean	false ▾
Object references (e.g., String, arrays)	null ▾

3.3.4 Analogy “Sticky Note on Cup of Coffee”

Imagine in a cafe there are 10 cups of coffee that look exactly the same. Attributes are like **sticky notes on cups of coffee**. Without attributes, the barista would be confused about which one is for “Budi” and which one is for “Fahmi” (who ordered with less sugar). With attributes, every cup has a sticky note on it. One note says “Budi, Ice Chocolate, with Extra Sugar” and another says “Fahmi, Americano, with Less Sugar”. The sticky note is an attribute, it’s always attached to the object.

4. Method

4.1 Why Should I Understand This?



An object that only has data (name, price) is passive (called POJO/Plain Old Java Object). In a coffee shop app, the system must perform actions like calculating discounts, decreasing stock, or printing receipts. **Methods** make it possible to separate business logic from the main code. This makes our code easy to maintain, reusable, and encapsulated.

4.2 Formal Definition

A **method** is a block of organized code that is declared in a class. It has a name (**signature**), can receive input parameters, process them, and optionally return a value or you may know it as a **function**. A **method**, also called **behavior**, is an action that can be performed by an object instance of a class. Example of methods of a car:

- `turnOn()` : a method that turns on the car's engine
- `drive()` : makes the car go forward, backward, or turn left or right
- `brake()` : stops or slows down the car

Characteristic of Methods:

- **Name** : The method name should describe the action. For example, a car can `drive()`, `brake()`, `turnOn()`
- **Parameter** : Input data passed when the method is called. For example, `drive(String turn)` where `String turn` is a parameter
- **Return Type** : Specifies the type of data returned by the method. For example, `topSpeed()` might return an integer
- **Body** : Contains the code that will be executed when the method is called, it starts with `{` and ends with `}`

4.3 Concept Elaboration

4.3.1 Anatomy of a Method

The structure of a method is as follows : `[Access Modifier]`
`[Return Type] [Method name] (Parameter) { Body }`

A method has a return type, if it's `void`, it doesn't need to return any value. If it's a variable type, it needs to return a value. For example:

```
1 public void displayInfo(String name) {
2     System.out.println("Hallo, Nama saya " + name);
3 }
```



If it uses a variable type instead of `void`, the code will look like this:

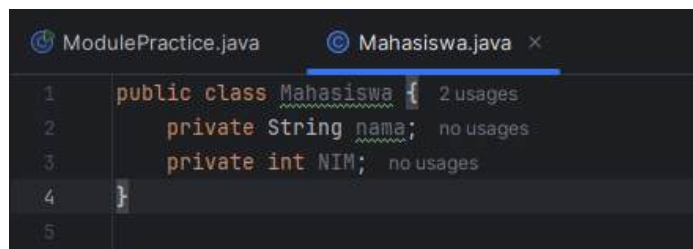
```
1 public int discount(int price, int percentage) {
2     int discountAmount = (price * percentage) / 100;
3     return price - discountAmount;
4 }
```

4.3.2 Method Overloading (Polymorphism Intro)

Imagine a barista received an order to make coffee. If the order is just to make coffee, it'll be plain black coffee (default). If the order was to make coffee and add milk (parameter), the barista will make a latte. The request is the same, make coffee, but the output is different because the parameters are different.

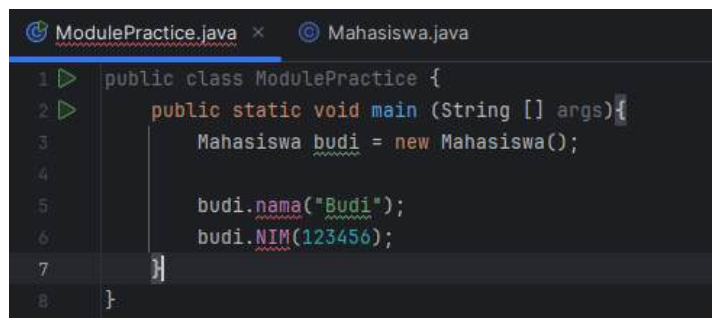
4.3.3 Getter Setter Method

In Chapter 2.3.5, we already discussed encapsulation. What if we have an attribute with a private access modifier?. How are we supposed to know or change the value of that attribute? We use **Getter and Setter** methods. Getter and setter methods are basically methods to get the value of an attribute and set the value of an attribute. For example:



```
1 public class Mahasiswa {
2     private String nama;
3     private int NIM;
4 }
5
```

We have a private `String nama` and a private `int NIM`. If we try to access or change them without getter setter like this:



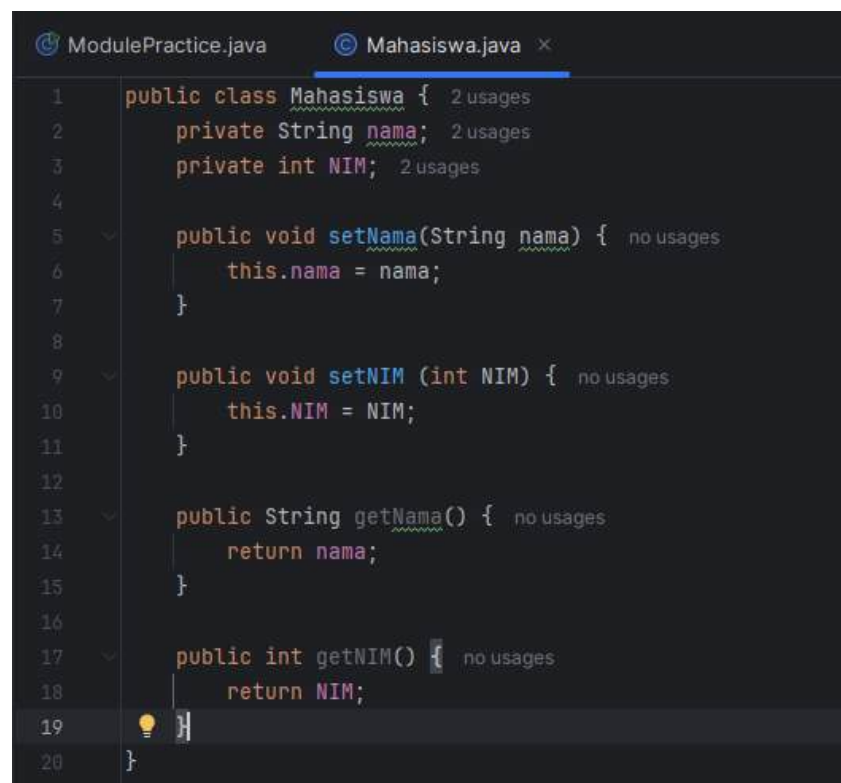
```
1 public class ModulePractice {
2     public static void main (String [] args){
3         Mahasiswa budi = new Mahasiswa();
4
5         budi.nama("Budi");
6         budi.NIM(123456);
7     }
8 }
```



If we try to run it, we'll get an error

```
D:\coding\Java\Modul PBO\CopasCode\src\ModulePractice.java:5:13
java: cannot find symbol
  symbol:   method nama(java.lang.String)
  location: variable budi of type Mahasiswa
```

This is why we need getters and setters to access private attributes. Here's how we create them:



```
ModulePractice.java  Mahasiswa.java x
1  public class Mahasiswa { 2 usages
2      private String nama; 2 usages
3      private int NIM; 2 usages
4
5      public void setNama(String nama) { no usages
6          this.nama = nama;
7      }
8
9      public void setNIM (int NIM) { no usages
10         this.NIM = NIM;
11     }
12
13     public String getNama() { no usages
14         return nama;
15     }
16
17     public int getNIM() { no usages
18         return NIM;
19     }
20 }
```



And here's how we use the methods:

```
ModulePractice.java x Mahasiswa.java  
1 public class ModulePractice {  
2     public static void main (String [] args){  
3         Mahasiswa budi = new Mahasiswa();  
4  
5         budi.setNama("Budi");  
6         budi.setNIM(123456);  
7  
8         System.out.println("Nama: " + budi.getNama());  
9         System.out.println("NIM: " + budi.getNIM());  
10    }  
11 }
```

And if we try to run it, it'll look like this:

```
Nama: Budi  
NIM: 123456  
  
Process finished with exit code 0
```



5. Static Method and Static Attribute

5.1 Why Should I Understand This?

In a coffee shop, there is data owned by the store, not by individual cups of coffee, for example, “total daily sales”. If we save it on each coffee object, then every cup of coffee would have its own record of sales, which is wrong. We need a variable that’s shared. That’s why we use `static`. Besides that, static is used for methods that don’t need an instance object (using less memory than creating instance objects).

5.2 Formal Definition

A **static method** and **static attribute** are methods or attributes that can be executed without creating an instance object. **Non-static** methods or attributes need an instance object to be used or called, and these are connected with OOP concepts.

A **static member** is a member of a class declared with the `static` keyword. This member is allocated in special memory (Method Area or Metaspace) when the class-loader loads the class, not when an object is created. A static member belongs to the class, not to instance objects.

5.3 Concept Elaboration

5.3.1 Instance vs Static Memory

Instance variables (non-static) are located on the Heap. Every object has its own copy (for example: `customerName`). A static variable is in a static memory, there’s only one copy in the application, no matter how many objects are created.

5.3.2 The Rule of Static

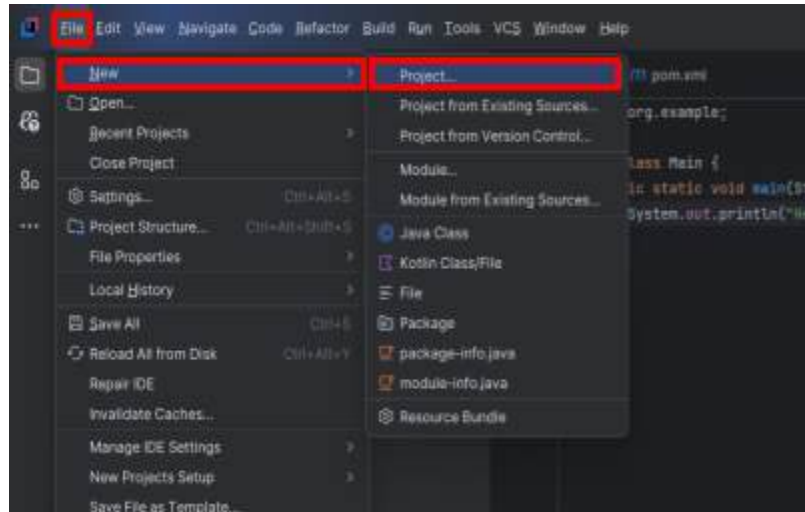
There’s some rule with static methods or attributes:

- **Shared state:** if one object changes the value of a static variable, all other objects will see the change.
- **Accessibility:** Static methods can be called without `new ClassName()`. You can simply use `ClassName.method()`.
- **Restriction :** Static Methods cannot directly access non-static variables. Why? Because static methods exist before any object is created. They don’t know which object instance you’re referring to.

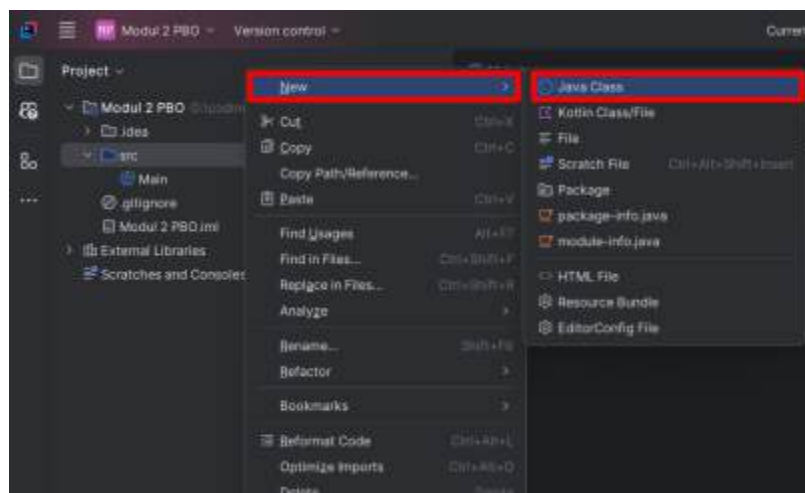


Practice

Let's try to create a class in IntelliJ IDEA. First create a new project: click on File > New > Project. Give it whatever name you want.



Right click on src > New > Java Class

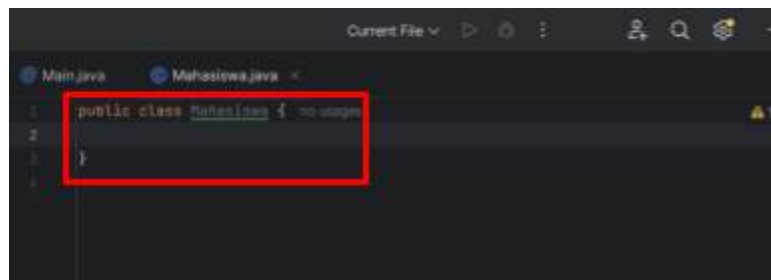


Give it the name "Mahasiswa" then press Enter

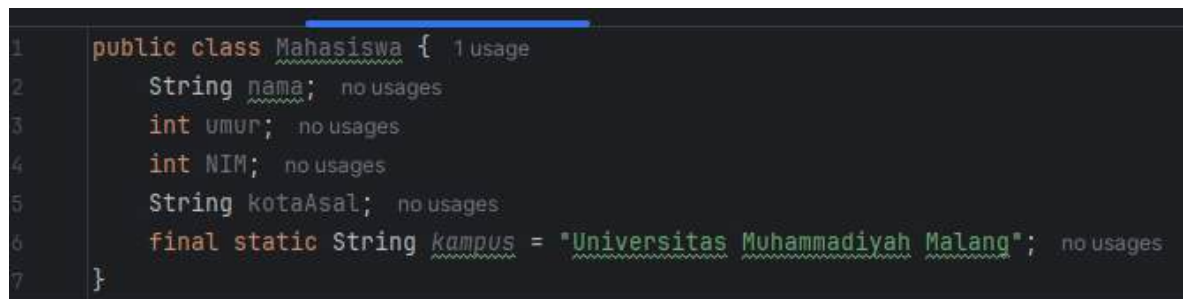




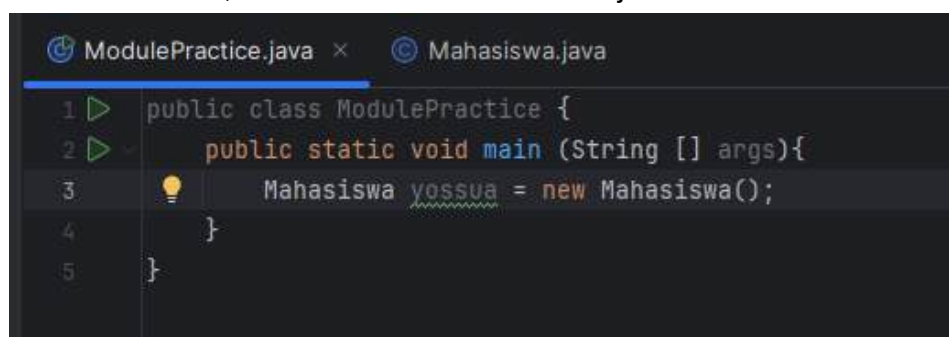
Now you have a class named "Mahasiswa". As you can see in the code here:



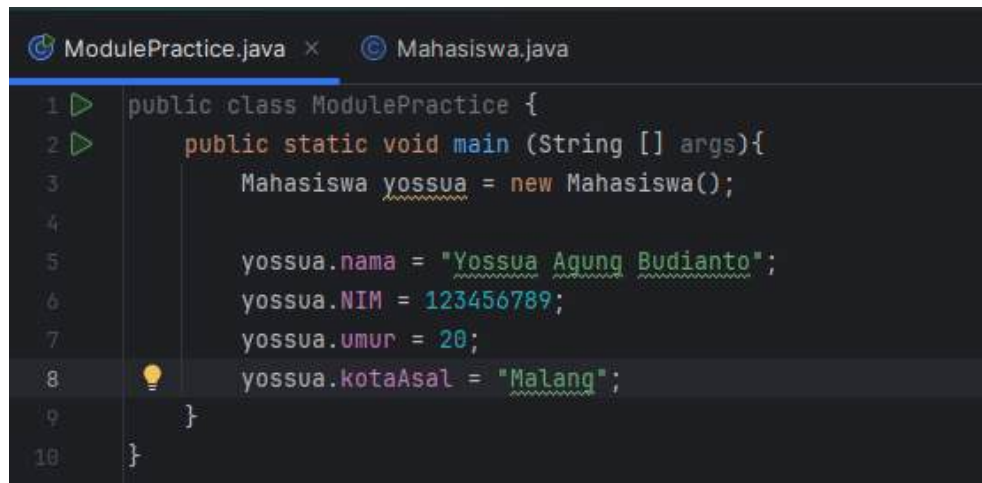
You can add attributes and methods as you want. For now, add a method like this:



In the main class, we can create an instance object:



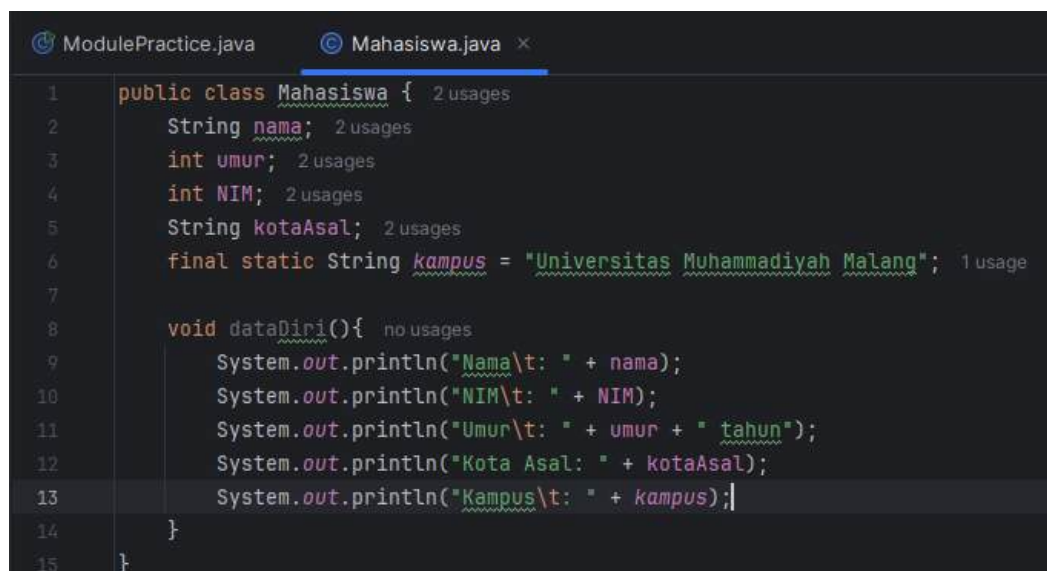
We can set attributes for this object:



```

ModulePractice.java x Mahasiswa.java
1 public class ModulePractice {
2     public static void main (String [] args){
3         Mahasiswa yossua = new Mahasiswa();
4
5         yossua.nama = "Yossua Agung Budianto";
6         yossua.NIM = 123456789;
7         yossua.umur = 20;
8         yossua.kotaAsal = "Malang";
9     }
10 }
  
```

Now, create a method to show details of our data:

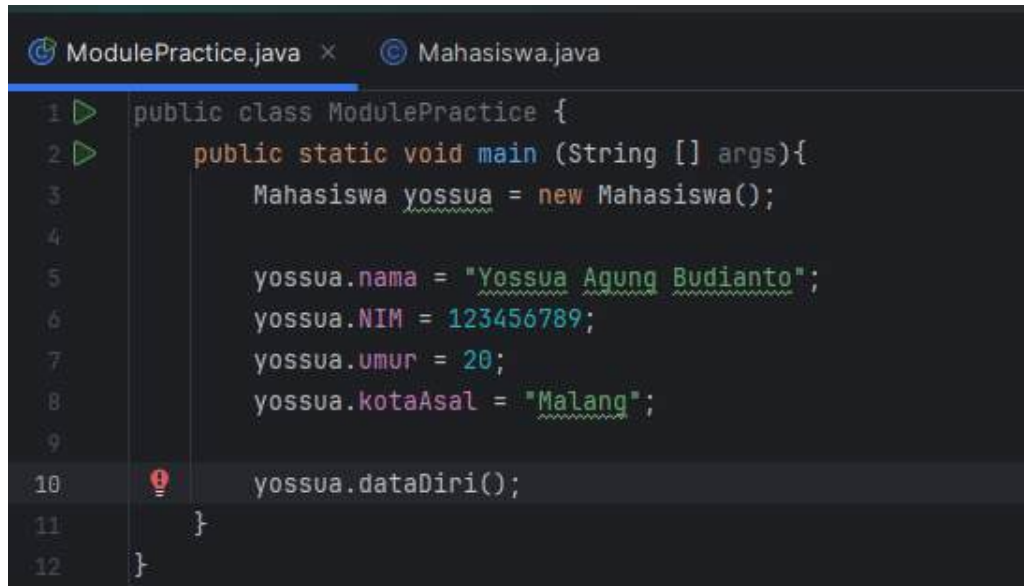


```

ModulePractice.java x Mahasiswa.java x
1 public class Mahasiswa { 2 usages
2     String nama; 2 usages
3     int umur; 2 usages
4     int NIM; 2 usages
5     String kotaAsal; 2 usages
6     final static String kampus = "Universitas Muhammadiyah Malang"; 1 usage
7
8     void dataDiri(){ no usages
9         System.out.println("Nama\t: " + nama);
10        System.out.println("NIM\t: " + NIM);
11        System.out.println("Umur\t: " + umur + " tahun");
12        System.out.println("Kota Asal: " + kotaAsal);
13        System.out.println("Kampus\t: " + kampus);
14    }
15 }
  
```



We can call the method to use the attribute:

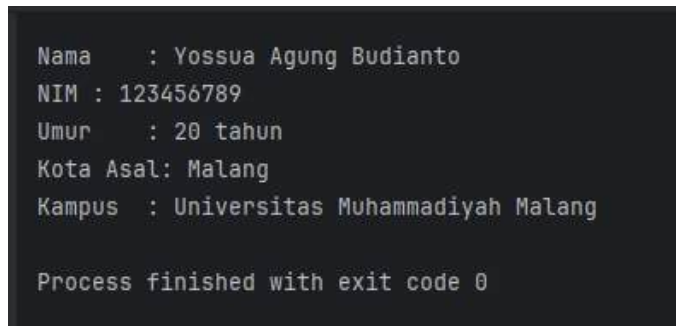


```

1  public class ModulePractice {
2      public static void main (String [] args){
3          Mahasiswa yossua = new Mahasiswa();
4
5          yossua.nama = "Yossua Agung Budianto";
6          yossua.NIM = 123456789;
7          yossua.umur = 20;
8          yossua.kotaAsal = "Malang";
9
10         yossua.dataDiri();
11     }
12 }

```

And the output will look like this:



```

Nama      : Yossua Agung Budianto
NIM : 123456789
Umur      : 20 tahun
Kota Asal: Malang
Kampus    : Universitas Muhammadiyah Malang

Process finished with exit code 0

```



Exercise

Let's continue the work from before

1. Add new class (create a new file named Character.java)
2. In Character.java, include an attributes and methods
3. Attributes:
 - a. Name
 - b. Age
 - c. heroClass (Archer/Mage/Warrior or etc)
 - d. Gender (Using boolean, true is for female *because female is always right :D*)
 - e. All methods must be a private
4. Methods:
 - a. createChar ()
After setting the attributes, call the displayInfo () method.
 - b. Getter Setter
Add getter and setter methods for all attributes

Characters.java

```

public class Character {
    String name; //Private
    int age; //Private
    String heroClass; //Private
    boolean gender; // Private // true = female cuz female always right :D

    public void createChar (String name, int age, String heroClass, boolean gender){
        this.name = name;
        this.age = age;
        this.heroClass = heroClass;
        this.gender = gender;

        displayCharInfo();
    }

    public void displayCharInfo() {
        System.out.println("====Character Information====");
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Gender: " + getGender());
        System.out.println("Class: " + heroClass);
    }

    //TODO: add method Getter Setter

```



Main.java

```
Character playerCharacter = new Character();  
playerCharacter.createChar(nameChar,ageChar,heroClass,gender);  
System.out.println("Character creation complete! Get ready for your adventure!");
```

Here's what you should understand:

1. What is a Character?
2. What is the difference between a class, method and attribute?
3. How to call a method from a class?
4. Can we call an attribute instead of a method from a class?
5. What is the variable used for the character class in your code?



Assignment

The mayor wants to upgrade our data collection system. The old system is messy and hard to manage, so you are required to rebuild the program using basic OOP concepts.

Here's what you should do:

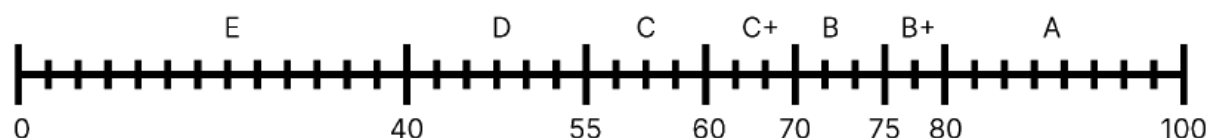
1. Create a new class named Building
2. The attribute of the class must include:
 - a. Name of the building
 - b. Address of the building
 - c. Number of floors
3. Add the following methods:
 - a. A method to display the data from the class attributes
 - b. A method to add a new building
4. In Main.java, create a class variable using the Building class and initialize it with attributes



Assessment

ASSESSMENT ASPECT	POINT
Codelab	Total 0%
Assignment	Total 100%
Code Running Smoothly	20%
Creativity Code	5%
Originality of the code	5%
Code Explanation	70%
Total	100%

Percentage Assessment



- A** = (81 - 100) → **You're The Chosen One**
- B+** = (75 - 80) → **Great!**
- B** = (70 - 74) → **Good!**
- C+** = (60 - 69) → **Keep it up!**
- C** = (55 - 59) → **Almost there!**
- D** = (41 - 54) → **Study Some More!**
- E** = (0 - 40) →



Closing Statement?



In a world before the Class was found,
Where all the code lay on the ground,
The Programmer wept, with aching head,
"A thousand movies! I'll be dead!"

For every film, a variable new,
A different name, a price, what to do?
String movie1, int price1... oh, the pain!
A manual task, completely insane.

Then OOP arrived, with a mighty decree,
"Let's build with Objects, and set your code free!"

